

ML Project 3 PDF

1. Experimental Setup

For this project, I trained and evaluated both MLPs and CNNs using PyTorch and torchvision. All experiments were run with CUDA on an NVIDIA GeForce RTX 3070. To keep the runs reproducible, I fixed the random seeds using `torch.manual_seed(42)` and `np.random.seed(42)`.

The two datasets used were MNIST and CIFAR-10. Both were loaded from `torchvision.datasets`. For preprocessing, I normalized each dataset using standard values. MNIST used mean 0.1307 and standard deviation 0.3081, while CIFAR-10 used mean (0.4914, 0.4822, 0.4465) and standard deviation (0.2023, 0.1994, 0.2010).

I split each dataset into training and validation sets before choosing final models. For MNIST, I used 50,000 samples for training and 10,000 for validation, with the original 10,000-image test set kept separate. For CIFAR-10, I used 45,000 samples for training and 5,000 for validation, again keeping the 10,000-image test set untouched until the end.

My overall training process was straightforward. I first used the train/validation split for hyperparameter tuning. After finding the best configuration based on validation accuracy, I retrained that final model on the combined train and validation data, then reported performance on the held-out test set. During tuning, I also used early stopping with patience set to 2 so I would not waste time training configurations that had clearly stopped improving.

2. Architectures Tested

2.1 MLP Architectures

For the first part of the project, I tested three different multilayer perceptrons.

Shallow MLP with one hidden layer containing 128 units. It used ReLU activation, dropout, and a final logits output layer.

Medium MLP with three hidden layers of sizes 512, 256, and 128. ReLU and dropout were used between layers, followed by a logits output layer.

Deep MLP with five hidden layers: 512, 256, 128, 64, and 32 units. Like the other MLPs, it used ReLU activations, dropout, and logits at the output.

For these MLPs, I tuned the following hyperparameters:

- Learning rate: {0.01, 0.001, 0.0001}
- Batch size: {32, 64, 128}
- Optimizer: {SGD, Adam}
- Dropout: {0.2, 0.5}

I did not run every possible combination exhaustively. Instead, I tested a meaningful subset of around 10 configurations per architecture.

2.2 CNN Architectures

For the CNN portion, I tested two different convolutional models.

The first was a simple CNN with two convolutional layers. Each layer used 8 filters with 3x3 kernels, stride 1, and padding 1. Batch normalization was applied after each convolution, and max pooling was used after each convolution block. A final fully connected layer handled classification.

The second was an enhanced CNN with three convolutional layers. This version increased the number of filters through the network: 16 -> 32 -> 64. Like the simple CNN, it used 3x3 kernels, stride 1, padding 1, batch normalization, pooling after each block, and a final fully connected layer for classification.

For the CNNs, I tuned:

- Learning rate: {0.01, 0.001, 0.0001}
- Batch size: {32, 64, 128}
- Weight decay: {0, 1e-4, 5e-3}

For these experiments, the optimizer was fixed to Adam. In total, I explored around 10 to 20 meaningful CNN configurations, with 12 being used here.

3. Hyperparameter Validation Summary

The project specification asks for validation accuracy with standard deviation, but my saved logs were not equally detailed across every experiment.

For CIFAR-10 MLPs, I still had the top three saved validation scores for each architecture, so I used those to estimate mean validation accuracy and standard deviation.

For CNNs and MNIST MLPs, I did not preserve repeated-run or per-configuration validation logs in a way that let me compute a clean standard deviation afterward. Because of that, those entries should be treated as single-run summaries rather than repeated-trial statistics.

3.1 CIFAR-10 MLP Validation Results

Architecture	Top-3 Validation Accuracies (%)	Mean Val Acc (%)	Std Dev (%)
MLP (shallow)	49.84, 49.42, 48.90	49.39	0.47
MLP (medium)	52.22, 49.16, 48.26	49.88	2.08
MLP (deep)	48.80, 47.06, 44.70	46.85	2.06

From these results, the medium MLP had the highest peak validation accuracy at 52.22%. However, it also showed more variation across its top runs than the shallow model. The shallow MLP was a little more stable overall, but it also hit a lower ceiling. The deep MLP performed the worst on CIFAR-10 in this tuning window, which most likely came from optimization difficulty and not having enough training budget to fully benefit from the extra depth.

4. Final Test Results

4.1 MNIST Results for MLPs

Architecture	LR	Batch	Opt	Dropout	Test Acc	Runtime
MLP (shallow)	0.001	64	Adam	0.2	97.85%	78.69s
MLP (medium)	0.001	64	Adam	0.2	98.15%	86.48s
MLP (deep)	0.001	64	Adam	0.2	97.81%	92.54s

On MNIST, all three MLPs performed well. The medium MLP ended up slightly ahead with 98.15% test accuracy. The shallow and deep versions were both close, but neither improved enough to justify saying they were clearly better.

4.2 CIFAR-10 Results for MLPs

At the moment, my saved results include the best validation results for CIFAR-10 MLPs, but the final test accuracies are still missing from the results file. Because of that, I am reporting the selected configurations and best validation accuracies below, while marking the final test accuracy as pending.

Architecture	LR	Batch	Opt	Dropout	Best Val Acc	Test Acc
MLP (shallow)	0.001	64	Adam	0.2	49.84%	Pending
MLP (medium)	0.0001	32	Adam	0.2	52.22%	Pending
MLP (deep)	0.0001	32	Adam	0.2	48.80%	Pending

Even without the final test values, the validation results already make it clear that the medium MLP was the strongest MLP architecture on CIFAR-10 out of the three tested.

4.3 MNIST Results for CNNs

Architecture	LR	Batch	Weight Decay	Test Acc	Runtime
CNN (simple)	0.01	64	0	98.38%	22.04s
CNN (enhanced)	0.01	128	0	98.80%	15.08s

Both CNNs did very well on MNIST, but the enhanced CNN performed best with 98.80% test accuracy. It also had the shortest runtime out of the two final MNIST CNN runs shown here.

4.4 CIFAR-10 Results for CNNs

Architecture	LR	Batch	Weight Decay	Test Acc	Runtime
CNN (simple)	0.001	64	0.005	61.85%	18.60s
CNN (enhanced)	0.001	32	0	73.96%	41.49s

On CIFAR-10, the difference between the two CNNs was much bigger. The enhanced CNN reached 73.96% test accuracy, while the simple CNN reached 61.85%. That is a large enough gap to clearly show that the more expressive CNN architecture was much better suited for CIFAR-10.

5. Discussion and Analysis

One of the main things I noticed was that **learning rate mattered a lot more** for some settings than others. In several of the MLP experiments, especially on CIFAR-10, the smaller learning rates like 0.001 and 0.0001 gave more stable training and better final validation accuracy than 0.01. This makes sense because CIFAR-10 is a harder dataset, and aggressive updates can make optimization less stable, especially for deeper fully connected networks. On the other hand, for CNNs on MNIST, 0.01 still worked well because MNIST is simpler and CNNs already have a strong structural advantage for image tasks.

Another clear pattern was that **architecture mattered more** than depth alone. On MNIST, almost every model worked reasonably well, including all three MLPs. The medium MLP slightly outperformed the shallow and deep ones, but the differences were small. That suggests MNIST is simple enough that once the model has enough capacity, adding more layers does not help much. In contrast, the CNNs did better overall, especially the enhanced CNN, which shows that convolutional structure is still more effective for image recognition even on an easier dataset.

For CIFAR-10, the gap between MLPs and CNNs was much larger. This was probably the most important result in the project. CIFAR-10 has more complex images, more variation, and more spatial structure than MNIST, so a plain MLP has a harder time

learning useful features efficiently. The CNNs handled this much better because they can build up local features across layers. That is why the enhanced CNN reached 73.96% while the simple CNN only reached 61.85%, and why both CNNs were clearly ahead of the MLP validation results.

The enhanced CNN did better than the simple CNN mostly because it had more representational power. Increasing the number of filters from 16 to 64 across layers gave the network more room to learn richer feature hierarchies. That mattered a lot for CIFAR-10, where the model needs to detect more complicated patterns than just clean digit shapes.

I also think the pooling strategy helped keep the CNNs practical. Pooling after each convolution block reduced spatial size and computation, which improved runtime and probably helped regularization too. At the same time, there is always a tradeoff, since too much downsampling can remove detail. In this case, though, the setup worked well enough to support strong performance, especially in the enhanced model.

Overall, the results followed a pretty intuitive pattern: **MNIST was easy enough that many architectures worked well, while CIFAR-10 separated the weaker models from the stronger ones much more clearly. The best-performing models were the ones whose structure matched the task better, not just the ones with the most layers.**

6. Key Challenge and How I Resolved It

The biggest issue I ran into during this project was **runtime during hyperparameter tuning**.

At first, training was taking much longer than I wanted, especially once I started testing many configurations across multiple architectures and two datasets. A lot of the slowdown came from repeated CPU-side preprocessing and repeated CPU-to-GPU transfers every mini-batch. Once that overhead stacked across many runs, tuning became much slower than it needed to be.

To fix this, I changed the pipeline to be more GPU-first. I pre-cached dataset tensors directly on the GPU, enabled cuDNN benchmark optimization, and cut tuning into a faster ranking phase so I could identify strong candidates without overtraining every single configuration. I also kept early stopping in place so weak runs would terminate quickly.

These changes made a major difference. Training and tuning became much more manageable, and I was able to explore enough configurations to make meaningful comparisons without the project becoming too slow to finish.

7. Final Model Choices

Based on the results I have available, these were the best final models from the project:

- Best MNIST MLP: MLP (medium), 98.15% test accuracy
- Best MNIST CNN: CNN (enhanced), 98.80% test accuracy
- Best CIFAR-10 CNN: CNN (enhanced), 73.96% test accuracy
- Best CIFAR-10 MLP by validation: MLP (medium), 52.22% validation accuracy, with final test accuracy still pending in the saved results

If I had to summarize the project in one sentence, it would be that CNN structure mattered much more than just increasing MLP depth, especially once the dataset became more complex.

8. Reproducibility Notes

To make the experiments as reproducible as possible, I used fixed random seeds, clearly defined train/validation/test splits, and documented the architecture choices and hyperparameter search ranges used in tuning.

One limitation is that not all repeated validation logs were preserved in the saved results, so standard deviation values could only be estimated cleanly for the CIFAR-10 MLP section. Also, the CIFAR-10 MLP test-set metrics are still missing from the current saved results file, so those entries remain marked as pending unless the models are rerun before submission.